menler

CLAUDE-FIRST FELLOWSHIP · WEEK 4

SOLUTION

My automated AI system

A reference build of an AI Student Assistant — it answers course questions by voice, logs every question to Notion, and emails a weekly digest. Yours can differ.

Week 4 · Claude · N8N · Notion · Airtable · Slack · VAPI

Reference build — yours can differ.

W4

My automated AI system

How it's executed

THE EXECUTION — HOW, WHERE & WHEN

The worked system is an **AI Student Assistant** for a course cohort. A student asks a question — by typing or by voice. Claude reads it, decides whether it is a **content question** (answer it), a **logistics question** (answer + flag), or **out of scope** (escalate to a human). It logs every question to Notion, and every Friday a Routine emails the instructor a digest of what was asked.

The **flow** lives in N8N. The **brain** is Claude, called once per question with a strict instruction. The **hands** are three MCP tools — Notion to log, Airtable to read the course FAQ, Slack to escalate. The **voice** is VAPI, which turns a phone call into text for Claude and Claude's answer back into speech.

WHAT TOOLS — AND THEIR ROLE

Claude	The intelligence. Classifies each question, answers from the FAQ, and decides when to escalate. Called once per question with a STAR instruction.
N8N	The skeleton. Receives the input, calls Claude, routes on its decision, and fans out to the tools.
Notion (MCP)	The memory. Every question and answer is logged as a row, so nothing is lost and the digest has data.
Airtable (MCP)	The knowledge. Holds the course FAQ that Claude reads before answering.
Slack (MCP)	The escape hatch. An out-of-scope question posts to the instructor channel.
VAPI	The voice layer. Speech-to-text in, Claude in the middle, text-to-speech out.

THE WORKED SOLUTION

Do it — *step by step (1 of 2)*

1 Design the agent brain

DO THIS

- Write four lines before you touch a tool: **Goal** (answer course questions, log them, escalate the rest), **Tools** (Notion, Airtable, Slack), **Memory** (the Notion log), **Escalation** (anything out of scope → Slack).
- Save this as your **agent brief**. It is the spec every later step points back to.

Done when: you can describe the system in four lines without hand-waving.

Tip — if a step is hard to build later, the brief was vague — fix the brief, not the wiring.

2 Write the Claude step in STAR

DO THIS

- Tell Claude exactly what to do and what shape to return. Strict JSON means N8N can route on it.
- Give it the FAQ text and the question, and demand one of three decisions.

CLAUDE STEP — INSTRUCTION

```
You are a course assistant. Read the FAQ and the
student question. Return ONLY JSON:
{ "type": "content" | "logistics" | "out_of_scope",
  "answer": "",
  "escalate": true | false }
```

Done when: the same question always returns the same clean JSON shape.

3 Build the 5-step Routine

DO THIS

- In Claude, create a Routine that chains the job: **receive** question → **read** FAQ (Airtable) → **decide + answer** (Claude) → **log** to Notion → **escalate** if needed (Slack).
- Name each step in plain words so you can see the flow at a glance.

Done when: running the Routine once walks all five steps with a test question.

THE WORKED SOLUTION

Do it — *step by step (2 of 2)*

4 Connect the three MCP tools

DO THIS

- Customize → Connectors → add **Notion**, **Airtable** and **Slack**, sign in and Allow for each.
- In the Routine's chat, open **+** → **Connectors** and switch all three on so the steps can actually reach them.

Done when: a test run writes a real row to Notion and can post to Slack.

Safety — give each connector the narrowest access that works — one database, one channel. Never paste a password or key into the chat.

5 Wire it into N8N

DO THIS

- Trigger: a **Webhook** node that receives the question. Then an **HTTP / Claude** node that runs the STAR instruction.
- Add a **Switch** node on `type` → content, logistics, out_of_scope. Each branch calls the matching MCP action.

Done when: posting a test question to the webhook lands the right action every time.

6 Add voice, schedule & test

DO THIS

- Point **VAPI** at the same webhook: it sends Claude the transcribed question and speaks the answer back.
- Add a **Schedule** for the Friday digest, then test the whole thing with a clean question, a messy one, and an out-of-scope one.

Done when: a spoken question is answered aloud, logged, and the Friday digest arrives on its own.

YOUR OUTPUT — THE CLAUDE STEP INSTRUCTION

```
You are a course assistant for [COURSE].
Read the FAQ context and the student's question.
Decide the type, answer only from the FAQ, and escalate
anything you cannot answer confidently.

Return ONLY this JSON:
{ "type": "content" | "logistics" | "out_of_scope",
  "answer": "one short paragraph, or empty string",
  "escalate": true | false }
```

TOOL OVERVIEW

What you'll find *where*

Every surface this build uses, and the one job it does. If you get lost, come back here.

Connectors	Customize → Connectors. Where Notion, Airtable and Slack get linked once, then switched on per chat.
Routines	Left strip → the Routine you built. The five-step job, re-runnable and editable.
Schedules	Left strip → Scheduled. Where the Friday digest runs on a timer. Pause here to stop it.
N8N canvas	Your N8N workspace. The webhook, the Claude call, and the Switch that routes the decision.
VAPI dashboard	Your VAPI account. The phone number and the agent that speak to the same webhook.
Notion database	The course log. One row per question — the raw material for the digest.

MAKE IT REUSABLE

Connect it & *automate it*

A build you run once is a demo. These three surfaces turn it into something that works every week without you.

CONNECT A CONNECTOR

Where: Customize → Connectors → + → pick the app → sign in → Allow. Then in a chat, + → Connectors → switch it on.

1. Add Notion, Airtable and Slack.
2. Sign in and Allow — Claude never sees the password.
3. Switch each one on inside the Routine's chat.

For this project: the assistant logs to Notion, reads the FAQ from Airtable, and escalates to Slack — all through connectors, no copy-paste.

Give each the narrowest scope that works — one database, one channel.

CREATE A SKILL

Where: Customize → Skills → New skill. A saved instruction Claude reuses on command.

1. Save your STAR classify-and-answer instruction as a skill.
2. Name it so a teammate knows when to use it.
3. Call it from any chat instead of re-pasting.

For this project: "Classify & answer a course question" becomes one reusable skill — the brain of the assistant, portable to next term's course.

MAKE A SCHEDULE

Where: Left strip → Scheduled → New task → describe the job → pick the rhythm → Schedule.

1. Describe it: "every Friday 5pm, email the instructor a digest of this week's Notion log."
2. Pick **weekly, Friday**.
3. Pause or delete from the same screen when term ends.

For this project: the weekly digest runs itself. **When to run:** anything on a rhythm. **When to stop:** Scheduled → Pause.

PROMPT LIBRARY · STAR

Write it in *STAR*

Four levels of the same idea — Situation, Task, Action, Result. Start basic, grow into pro. Each is a complete prompt you can paste and adapt.

BASIC

- S **Situation.** I run a course and get the same questions over and over.
- T **Task.** Answer a student question from this FAQ.
- A **Action.** Read the FAQ, answer in one short paragraph, and say "ask the instructor" if it is not covered.
- R **Result.** A plain answer a student can act on, or a clean hand-off.

INTERMEDIATE

- S **Situation.** Questions arrive by text and by voice, and I want them all logged.
- T **Task.** Classify each question, answer it, and return a clean record.
- A **Action.** Return JSON with type, answer and an escalate flag, reading only from the FAQ.
- R **Result.** A structured result N8N can route and Notion can store as one row.

ADVANCED

- S **Situation.** Some questions are logistics, some are out of scope, and those need a human.
- T **Task.** Decide the type and route accordingly, escalating what you cannot answer.
- A **Action.** Classify into content / logistics / out_of_scope; set escalate:true for the last; never invent facts beyond the FAQ.
- R **Result.** Students get instant answers; the instructor only sees what truly needs them.

PRO

- S **Situation.** I want the whole thing to run and report itself every week.
- T **Task.** Answer and log all week, then summarise the week for the instructor.
- A **Action.** On each question, classify-answer-log; on a Friday Schedule, read the week's Notion rows and draft a digest of themes, volume and open escalations.
- R **Result.** A self-running assistant that also tells its owner what happened — zero manual work.

WHY IT MATTERS

Where this pays off *in real life*

The same four-box pattern — input → decide → act → deliver — is behind most of the AI systems people are paid to build.

- **Student assistant** — a course or college answers routine questions day and night, and staff only handle the real ones.
- **Startup intake** — inbound leads or support tickets get read, classified, logged to the CRM, and routed to the right person.
- **Content pipeline** — a topic in, and a draft, a summary and a scheduled post out, with a human approving the last step.
- **Research assistant** — a daily brief that gathers, filters and summarises on a Schedule before you wake up.

WHAT GREAT LOOKS LIKE

- The system runs end to end with **no manual step** in the middle.
- Claude makes a **real decision** that changes what happens next.
- Every run leaves a **record** you can read later.
- It fails **gracefully** — an unclear input escalates instead of guessing.